

MySQL Performance: e o que minha query tem a ver com isso?



Roberto de Bem
Database Engineer
Percona



Agenda

01

Quem sou eu?

02

O que é Performance?

03

North Star

04

Importância dos Índices
e da Indexação

05

Método de Acesso à Tabela

06

Índices e mais Índices

07

Pensando como MySQL

08

Thank you

Quem sou eu?



Quem sou eu?

- Database Engineer – Percona
- 7 anos trabalhando com Banco de Dados
- Formado em Sistemas de Informação – UNISUL
- Oracle ACE Associate
- MySQL 8.0 Database Administrator – OCP



LinkedIn



Créditos

Essa palestra é baseada no Livro "Efficient MySQL Performance: Best Practices and Techniques" de Daniel Nichter

<https://hackmysql.com/>

O que é Performance?

Query Response Time

Quanto tempo o MySQL leva para executar uma query

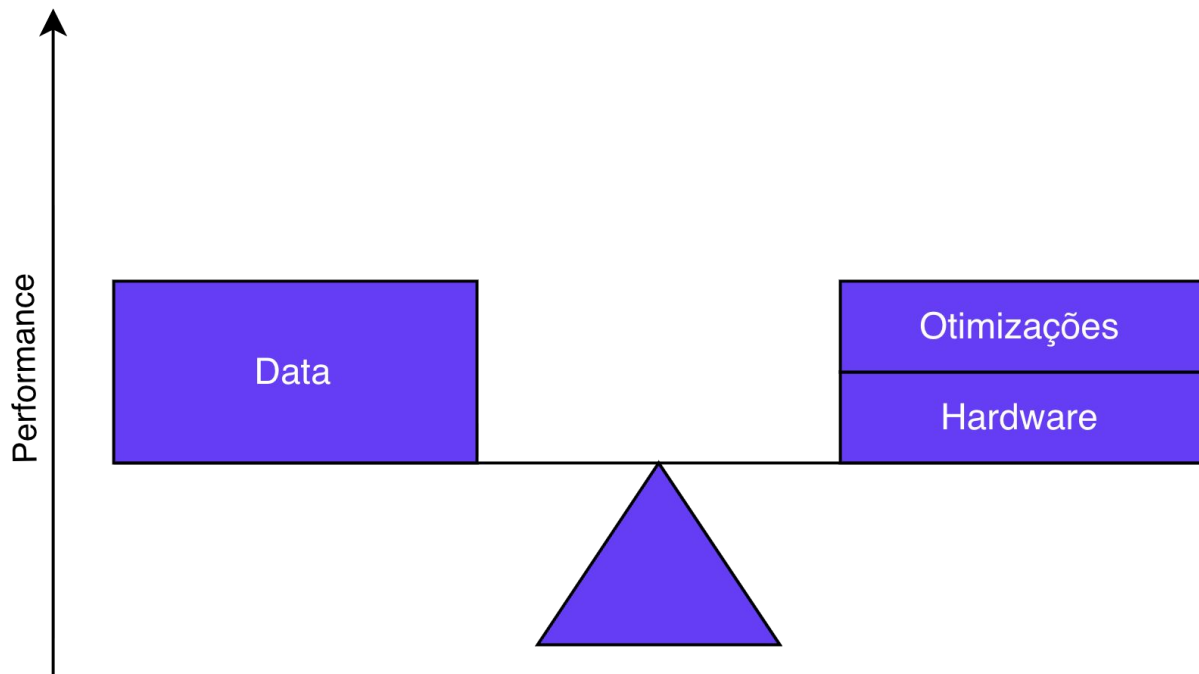
North Star

- **MySQL não faz nada**
- **Métrica percebida**
- **Tempo de resposta**

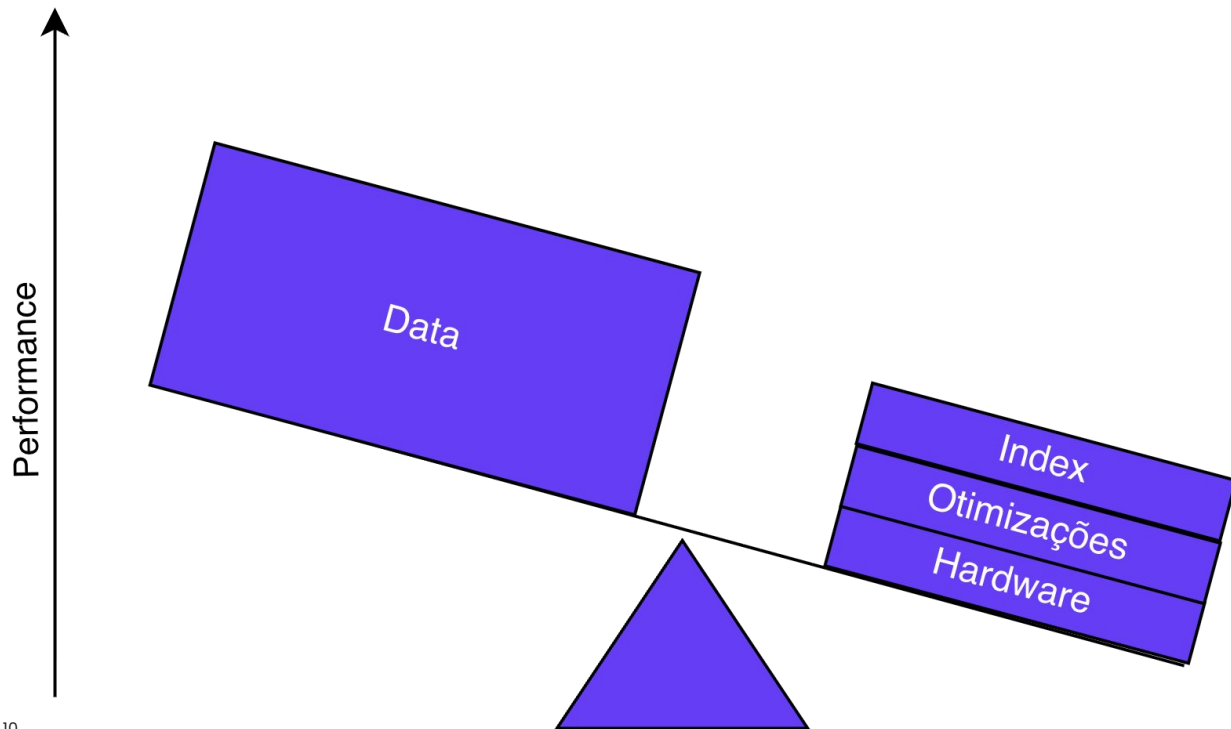
Importância dos Índices e da Indexação



▷ Performance sem Índices



▷ Performance com Índices



▶ Tabelas da apresentação

```
mysql> SHOW CREATE TABLE chemical_elements\G
***** 1. row *****

Table: chemical_elements
Create Table: CREATE TABLE `chemical_elements` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(50) NOT NULL,
  `acronym` char(2) NOT NULL,
  `atomic_number` int NOT NULL,
  `description` varchar(700) NOT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_composite` (`acronym`,`atomic_number`,`name`,`description`),
  KEY `idx_acro_desc` (`acronym`,`description`),
  KEY `idx_atomic_name_desc` (`atomic_number`,`name`,`description`),
  KEY `idx_description` (`description`(200)),
  KEY `idx_func_protons` ((concat(`atomic_number`,`_utf8mb4' protons')))
) ENGINE=InnoDB AUTO_INCREMENT=237 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

▶ Tabelas da apresentação

```
mysql> SHOW CREATE TABLE jointit\G
***** 1. row *****
      Table: jointit
Create Table: CREATE TABLE `jointit` (
  `s` varchar(64) NOT NULL,
  `t` time NOT NULL,
  `g` int NOT NULL,
  `h` varchar(500) DEFAULT NULL,
  PRIMARY KEY (`s`),
  KEY `idx_g` (`g`),
  KEY `idx_g_t` (`g`,`t`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
1 row in set (0.00 sec)
```

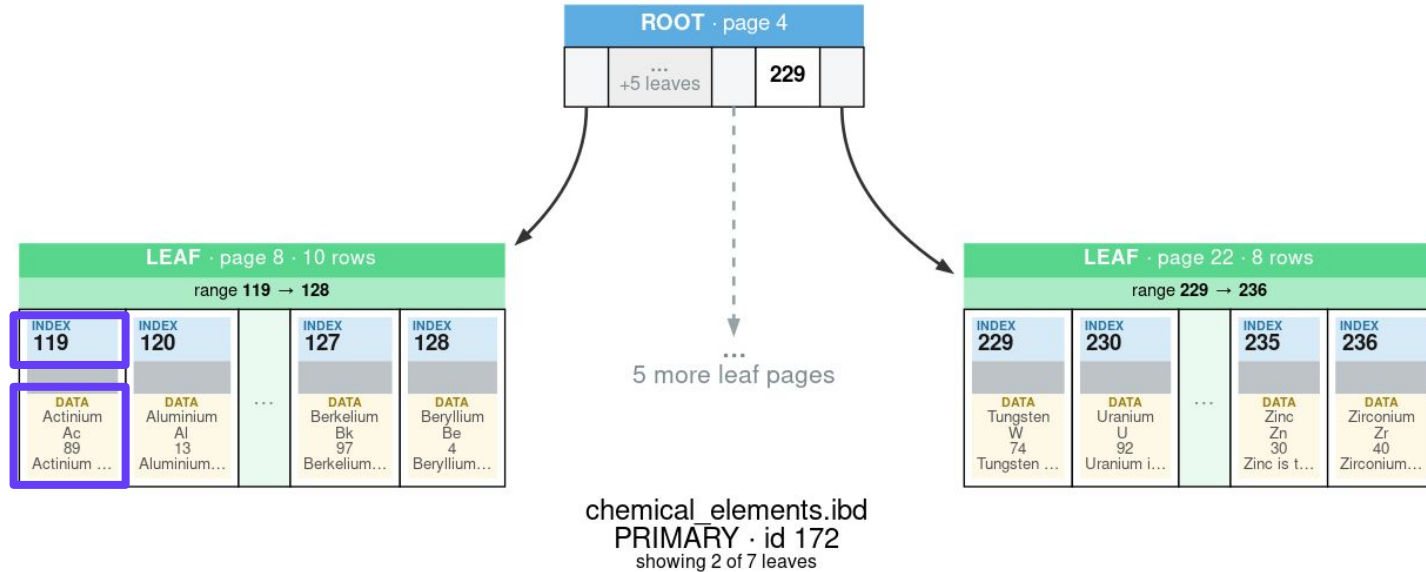
► Visualização da tabela

```
mysql> SELECT id,name,acronym,atomic_number FROM chemical_elements ORDER BY id LIMIT 10;
```

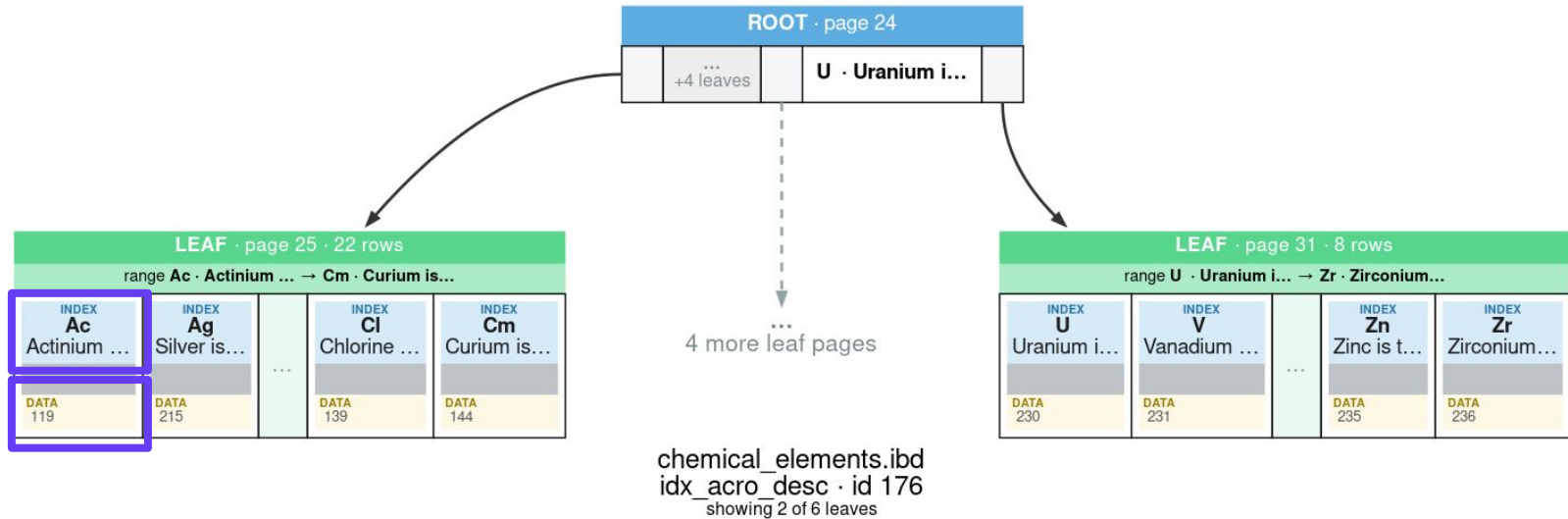
id	name	acronym	atomic_number
119	Actinium	Ac	89
120	Aluminium	Al	13
121	Americium	Am	95
122	Antimony	Sb	51
123	Argon	Ar	18
124	Arsenic	As	33
125	Astatine	At	85
126	Barium	Ba	56
127	Berkelium	Bk	97
128	Beryllium	Be	4

10 rows in set (0.00 sec)

Primary Key - Clustered index



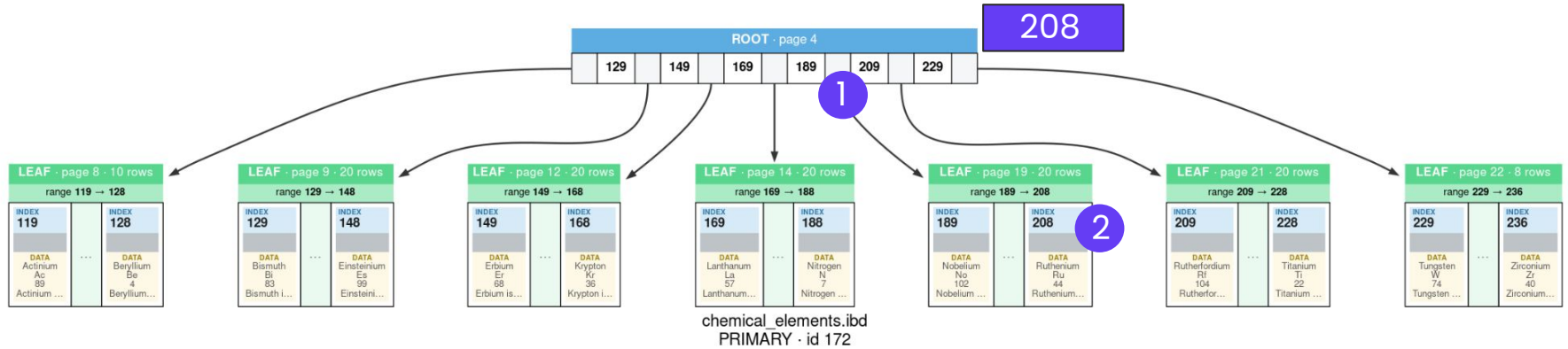
▶ Secondary index



Método de Acesso à Tabela



Primary Key Lookup

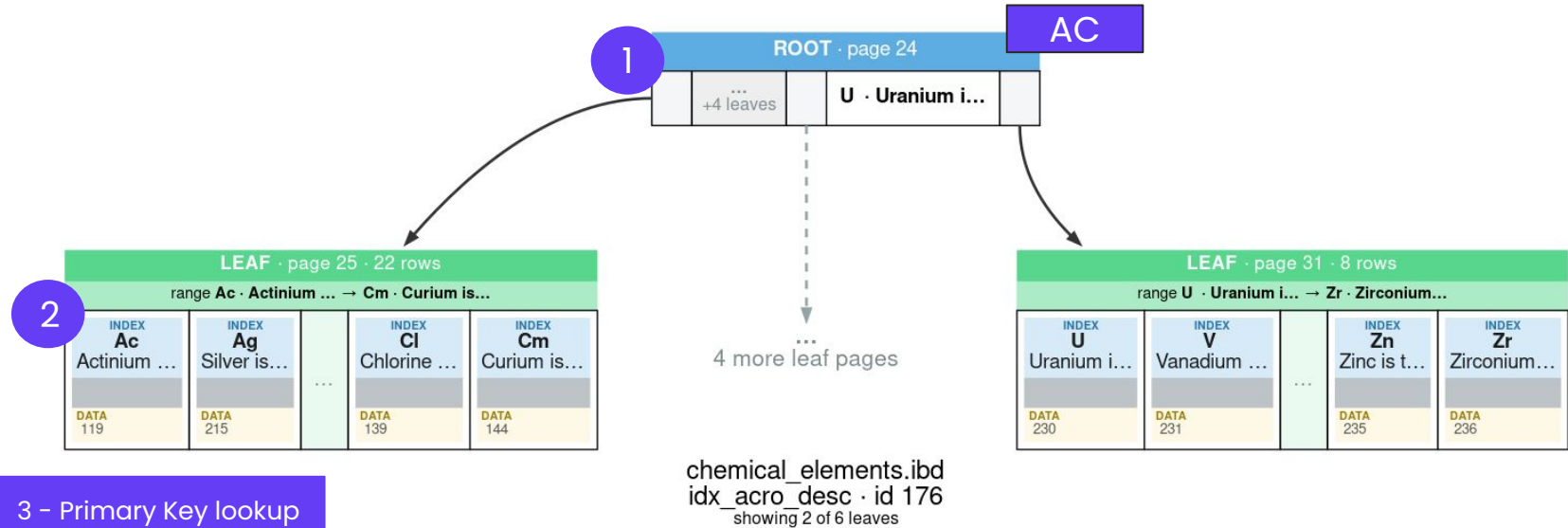


SELECT * FROM chemical_elements WHERE ID = 208;

Primary Key Lookup

```
mysql> EXPLAIN SELECT * FROM chemical_elements WHERE ID = 208\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: chemical_elements
  partitions: NULL
         type: const
possible_keys: PRIMARY
          key: PRIMARY
       key_len: 4
         ref: const
        rows: 1
   filtered: 100.00
      Extra: NULL
1 row in set, 1 warning (0.00 sec)
```

▶ Index lookup



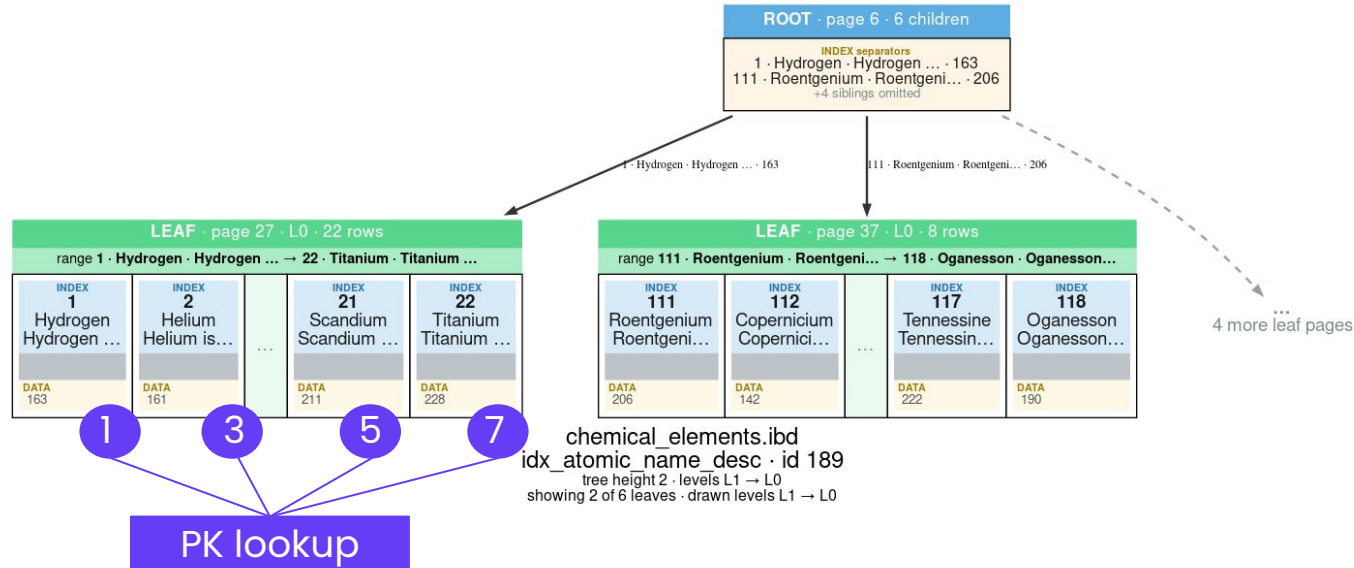
SELECT name FROM chemical_elements WHERE acronym = 'AC';

Index lookup

```
mysql> EXPLAIN ANALYZE SELECT name FROM chemical_elements WHERE acronym = 'AC'\G
***** 1. row *****
EXPLAIN: -> Index lookup on chemical_elements using idx_acro_desc (acronym='AC'), with index condition:
(chemical_elements.acronym = 'AC') (cost=0.35 rows=1) (actual time=0.0246..0.026 rows=1 loops=1)

1 row in set (0.00 sec)
```

Index Scan



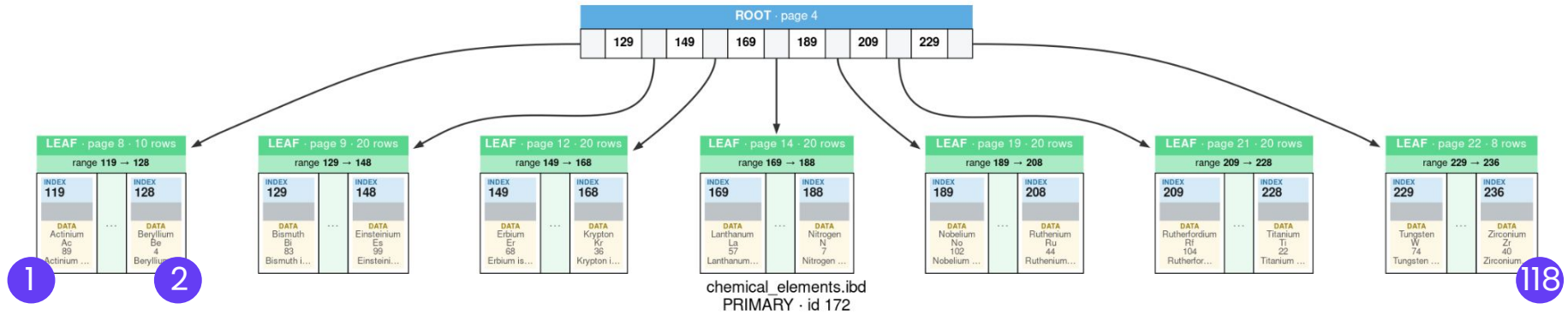
SELECT * FROM chemical_elements FORCE INDEX
(idx_atomic_name_desc) ORDER BY atomic_number,name;

Index Scan

```
mysql> EXPLAIN ANALYZE SELECT * FROM chemical_elements FORCE INDEX (idx_atomic_name_desc) ORDER BY atomic_number,name\G
***** 1. row *****
EXPLAIN: -> Index scan on chemical_elements using idx_atomic_name_desc (cost=41.5 rows=118) (actual time=0.0513..0.189 rows=118 loops=1)

1 row in set (0.00 sec)
```

▶ (Full) Table Scan



`SELECT * FROM chemical_elements;`

▶ (Full) Table Scan

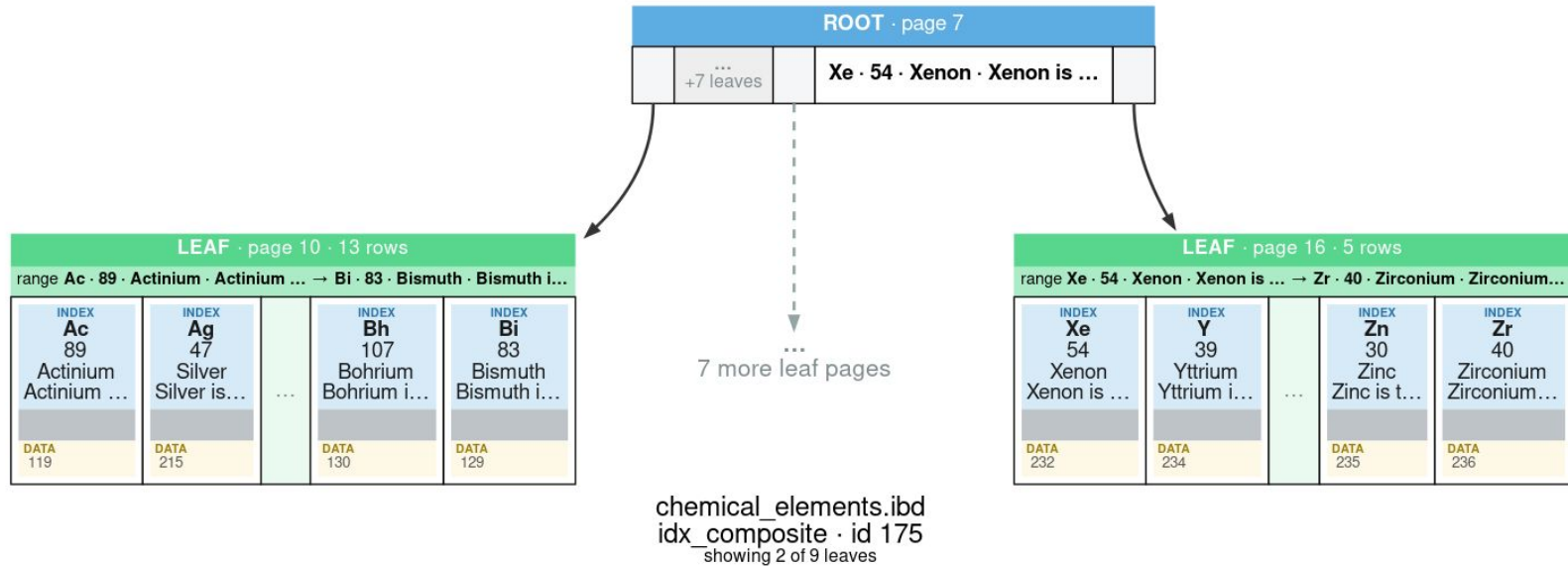
```
mysql> EXPLAIN ANALYZE SELECT * FROM chemical_elements\G
***** 1. row *****
EXPLAIN: -> Table scan on chemical_elements (cost=13.8 rows=118) (actual time=0.0334..0.0822 rows=118 loops=1)

1 row in set (0.00 sec)
```

**Índices e
mais Índices**

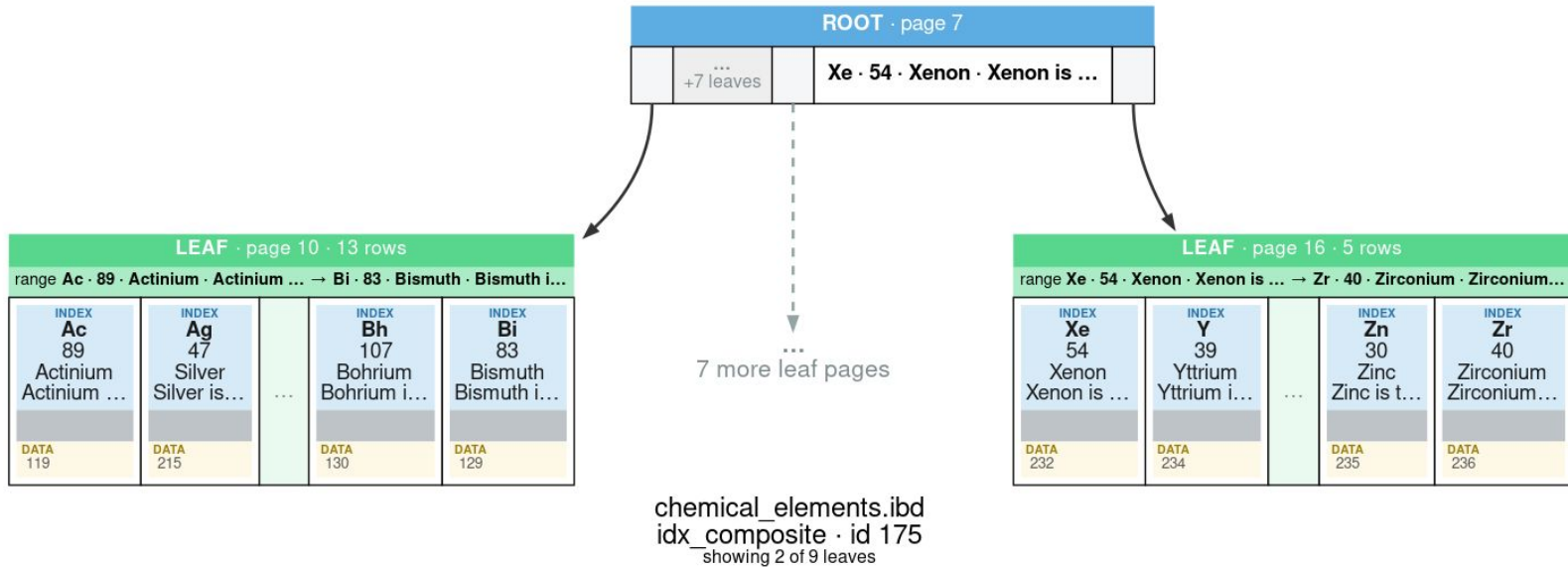


▶ Leftmost Prefix



SELECT * FROM chemical_elements WHERE atomic_number = 54;

Leftmost Prefix



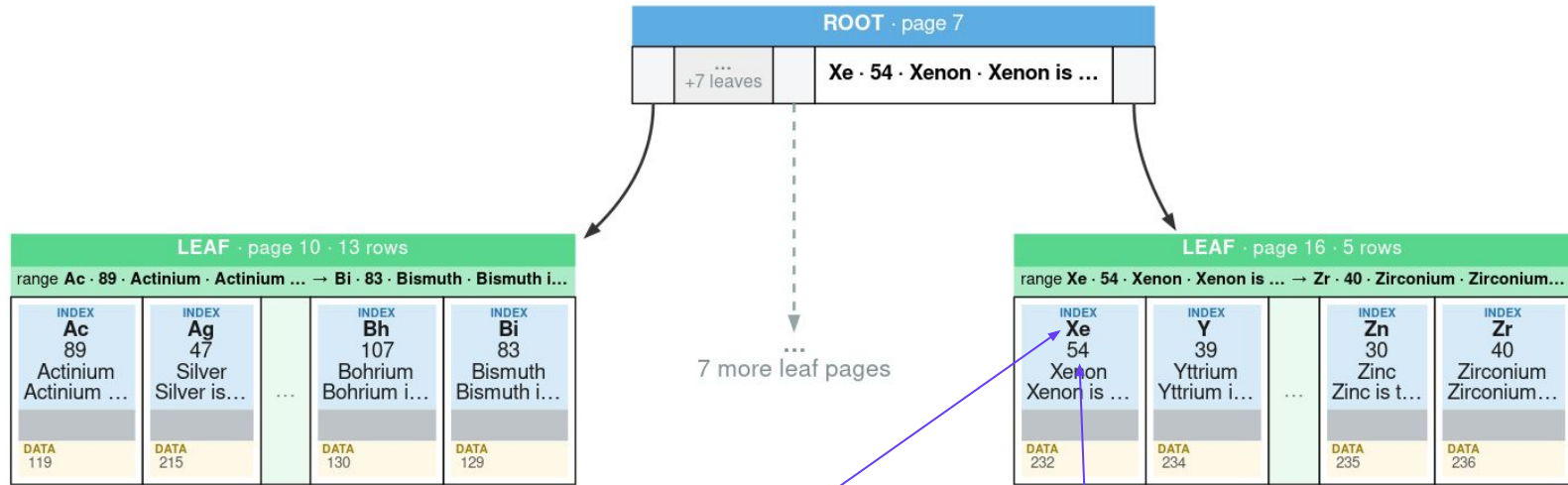
SELECT * FROM chemical_elements WHERE acronym = 'Bh';

▶ Leftmost Prefix

- Índice A,B e B,A são diferentes
- MySQL pode usar índice A,B,C invés do índice A,B ou A
- A,B e A são índices duplicados

```
KEY `idx_composite` (`acronym`,`atomic_number`,`name`,`description`),  
KEY `idx_acro_desc` (`acronym`,`description`),
```

▶ Covering Index



chemical_elements.ibd
idx_composite · id 175
showing 2 of 9 leaves

```
SELECT acronym, atomic_number FROM chemical_elements  
WHERE acronym = 'XE' and atomic_number=54;
```

Covering Index

```
mysql> EXPLAIN ANALYZE SELECT acronym, atomic_number FROM chemical_elements WHERE acronym = 'XE' and atomic_number=54\G
***** 1. row *****
EXPLAIN: -> Filter: (chemical_elements.acronym = 'XE') (cost=0.35 rows=1) (actual time=0.019..0.0212 rows=1 loops=1)
          -> Covering index lookup on chemical_elements using idx_composite (acronym='XE', atomic_number=54) (cost=0.35 rows=1) (actual time=0.01
          .0.0198 rows=1 loops=1)

1 row in set (0.01 sec)
```

Prefix Index

```
SELECT  
COUNT(DISTINCT description)/COUNT(*) as full_index,  
COUNT(DISTINCT LEFT(description,16))/COUNT(*) as 16c_idx,  
COUNT(DISTINCT LEFT(description,32))/COUNT(*) as 32c_idx,  
COUNT(DISTINCT LEFT(description,64))/COUNT(*) as 64c_idx  
FROM test \G
```

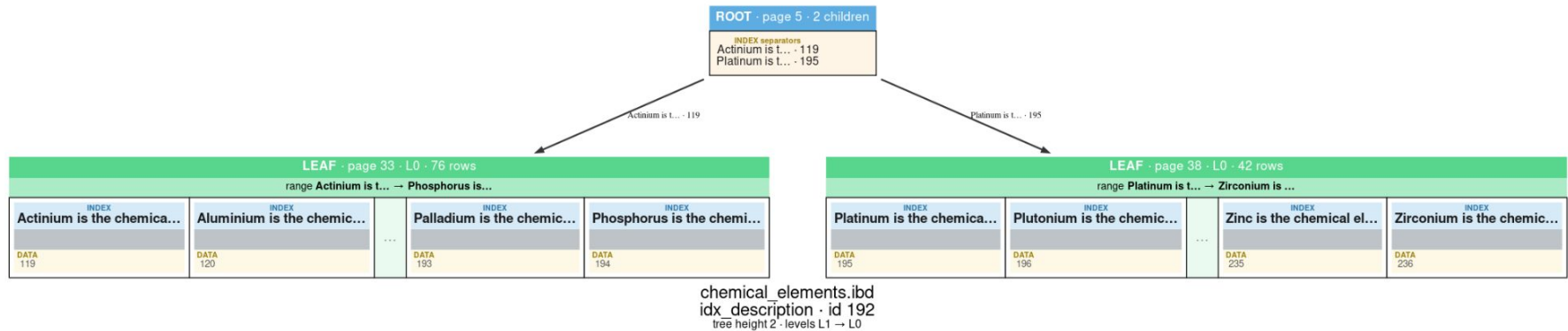
full_index: 0.9980

16c_idx: 0.5420

32c_idx: 0.9964 **# Index com menor tamanho 32 char**

64c_idx: 0.9980

▶ Prefix Index



`idx_description (description(200))`

Functional Indexes

```
mysql> ALTER TABLE chemical_elements  
-> ADD INDEX idx_func_protons ((CONCAT(atomic_number, ' protons')));  
Query OK, 0 rows affected (0.10 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> EXPLAIN ANALYZE SELECT * FROM chemical_elements WHERE ((concat(`atomic_number`,_utf8mb4' protons')) = '1 protons'\G  
***** 1. row *****  
EXPLAIN: -> Index lookup on chemical_elements using idx_func_protons (concat(atomic_number,_utf8mb4' protons')='1 protons') (cost=0.35 rows=1)  
(actual time=0.0288..0.0306 rows=1 loops=1)
```

```
1 row in set (0.00 sec)
```

▶ Functional Indexes

LEAF · page 39 · L0 · 118 rows				
range 1 protons → 99 protons				
INDEX 1 protons	INDEX 10 protons	...	INDEX 98 protons	INDEX 99 protons
DATA 163	DATA 183		DATA 135	DATA 148

chemical_elements.ibd
idx_func_protons · id 193
tree height 1 · levels L0

Group By

```
mysql> EXPLAIN SELECT g,t FROM joint WHERE t='17:59:22' GROUP BY g\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: joint
   partitions: NULL
         type: range
possible_keys: idx_g,idx_g_t
          key: idx_g_t
       key_len: 7
         ref: NULL
        rows: 62
   filtered: 100.00
      Extra: Using where; Using index for group-by
1 row in set, 1 warning (0.00 sec)
```

Order By

```
mysql> EXPLAIN SELECT * FROM jointit WHERE t='17:59:22'
-> AND G=50 ORDER BY t\G
***** 1. row *****
      id: 1
select_type: SIMPLE
      table: jointit
  partitions: NULL
        type: ref
possible_keys: idx_g,idx_g_t
          key: idx_g_t
        key_len: 7
          ref: const,const
         rows: 1
    filtered: 100.00
      Extra: Using filesort
1 row in set, 1 warning (0.00 sec)
```

```
mysql> EXPLAIN SELECT * FROM jointit WHERE t='17:59:22'
-> AND G=50 ORDER BY s\G
***** 1. row *****
      id: 1
select_type: SIMPLE
      table: jointit
  partitions: NULL
        type: ref
possible_keys: idx_g,idx_g_t
          key: idx_g_t
        key_len: 7
          ref: const,const
         rows: 1
    filtered: 100.00
      Extra: NULL
1 row in set, 1 warning (0.00 sec)
```

Ordem das colunas

```
mysql> show warnings\G
```

```
***** 1. row *****
```

```
Level: Note
```

```
Code: 1003
```

```
Message: /* select#1 */ select `test`.`joinit`.`s` AS `s`,`test`.`joinit`.`t` AS `t`,`test`.`joinit`.`g` AS `g`,`test`.`joinit`.`h` AS `h` from  
`test`.`joinit` where ((`test`.`joinit`.`g` = 50) and (`test`.`joinit`.`t` = TIME'17:59:22')) order by `test`.`joinit`.`s`
```

```
1 row in set (0.00 sec)
```

**Pensando
como MySQL**



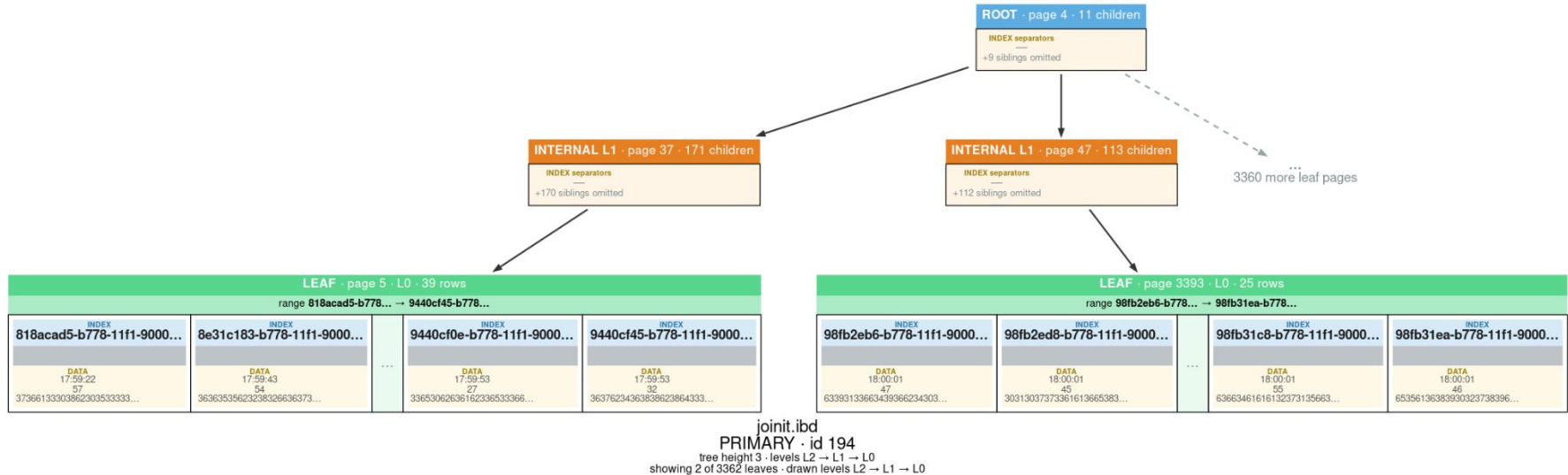
Pensando como MySQL

- Entenda a query
 - Quantas linhas essa query deveria acessar?
 - Quantas linhas essa query deveria retornar?
 - Quais colunas estou requisitando?
 - Há valores no Group By, Order By?
- Table Access
 - Qual index essa query deveria usar? Há somente um?
 - Qual a cardinalidade do index?
 - Quão grande é a tabela?

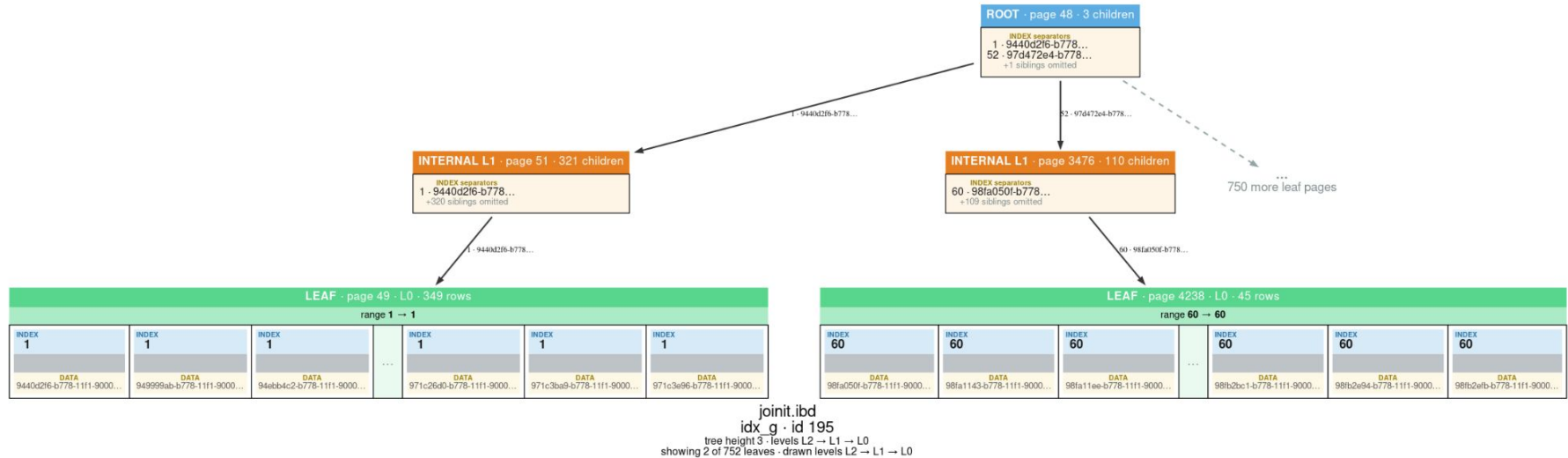
Era um bom Índice até que...

- A Query mudou
- Índice em excesso, duplicados
- Seletividade do Índice
 - Colunas verdadeiro ou falso (active). Active = 1
 - 10.000 Usuários - 9.000 ativos = seletividade de 0,0002%
 - Bom índice para inativos, péssimo para ativos
- Tabela cresceu

► Bonus – UUID como PK, é ruim?



► Bonus – UUID como PK, é ruim?



Perguntas?



LinkedIn



Slides

A mountain goat with white fur and small horns stands on a dark, rocky outcrop. The background is a vast, rugged mountain range with snow-dusted peaks under a clear, deep blue sky. The scene is illuminated by warm, low-angle light, likely from the setting or rising sun, creating a golden glow on the mountain's ridges and the goat's fur.

THANK YOU

